

Métricas y estimaciones

Apunte de la unidad

Prof. Sebastián Schanz · Facultad de Ingeniería, Sede Trelew · UNPSJB · 2026



1. Introducción

Esta unidad responde dos preguntas que cualquier proyecto se hace desde el primer día: cómo sabemos algo del software con números (cuán grande es, cuántos defectos tiene, qué tan productivo fue el equipo) y cuánto va a costar construirlo, en esfuerzo, plazo y dinero, cuando todavía no existe.

Las dos preguntas están atadas: para estimar hace falta tener con qué (datos de proyectos anteriores, una medida del tamaño de lo que se va a construir, la productividad del equipo), y esos datos sólo existen si alguien midió antes. Por eso el apunte tiene tres partes. La Parte I construye el vocabulario y los criterios de la medición. La Parte II plantea el problema de la estimación y recorre los métodos clásicos, incluidos los modelos empíricos y COCOMO. La Parte III desarrolla los Puntos de Función, que permiten medir el tamaño de un sistema antes de escribir una línea de código, con un ejemplo completo, y cierra con la estimación en equipos ágiles.

En la etapa 2 de la ISFPP van a estimar su sistema por Puntos de Función y cotizarlo; en la etapa 4 van a contar las líneas de código reales y compararlas con las estimadas. Conviene leer el apunte con esa comparación en mente.

PARA PENSAR ANTES DE SEGUIR

Contesten, a ojo y para su ISFPP: ¿cuánto va a durar?, ¿cuánto esfuerzo requiere?, ¿cuánta gente hace falta? Anoten los números y, sobre todo, de dónde los sacaron. Al final de la Parte II vuelvan a mirarlos.

PARTE I

Medir

Medición, métricas e indicadores · qué se mide en software · validez · métricas de tamaño, productividad y complejidad

2. ¿Se puede comparar sin medir?

2.1 Dos equipos

Empecemos con un caso. El equipo A encontró 342 errores antes de entregar su sistema. El equipo B encontró 184. ¿Cuál trabajó mejor?

Ninguna respuesta rápida se sostiene. No sabemos de qué tamaño era cada sistema: 342 errores en un sistema enorme pueden ser pocos, y 184 en uno chico pueden ser muchos. No sabemos cuántos errores quedaron sin encontrar. Y no sabemos cuánto tiempo llevó encontrarlos.

El caso (un ejercicio de Pressman) deja una primera lección: un número suelto no permite decidir nada. Empieza a decir algo cuando sabemos de qué es, contra qué se compara y respecto de qué se normaliza (el tamaño, el tiempo, el esfuerzo).

2.2 Cualitativo y cuantitativo

La descripción cualitativa dice cómo es algo («el código del equipo A está bastante prolijo»); la cuantitativa dice cuánto, con una unidad («el código del equipo A tiene 12 defectos cada mil líneas»).

Ninguna reemplaza a la otra. Lo cuantitativo permite comparar, ordenar y seguir en el tiempo; lo cualitativo permite entender por qué. Un número sin interpretación no sirve para decidir, y una impresión sin número no se puede discutir con nadie. Medir no reemplaza el criterio del ingeniero: le da con qué discutir.

3. Qué es medir

3.1 La definición

Norman Fenton define la medición como *el proceso mediante el que se asignan números o símbolos a los atributos de las entidades del mundo real, de manera que dichos atributos queden descritos de acuerdo a reglas previamente definidas* (Fenton, 1997). Se mide una característica de una cosa, no la cosa; y la regla es lo que garantiza que dos mediciones hechas por personas distintas se puedan comparar.

3.2 Entidad, atributo, medida

Toda medición tiene tres partes: la entidad (el objeto que se mide), el atributo (la característica que interesa) y la medida (el valor que resulta de aplicar la regla).

Entidad	Atributo	Medida
Persona	Altura	1,72 m
Vehículo	Velocidad	100 km/h
Programa	Tamaño	4.500 líneas de código

La consecuencia: no se mide «el software»; se mide un atributo del software. No se mide el auto sino su peso, su consumo o su distancia de frenado; no se mide el software sino su tamaño, su complejidad o sus defectos. La pregunta «¿qué atributo, de qué entidad?» destraba casi todos los problemas de medición. En el caso de los dos equipos, «342 errores» es una medida, pero ¿de qué

entidad? ¿Del sistema, del proceso de pruebas, del equipo? Según la respuesta, el número significa cosas distintas.

4. Medida, métrica e indicador

4.1 Tres palabras que no son sinónimos

En el habla cotidiana se usan como si fueran lo mismo, pero cada una responde una pregunta distinta:

- **Medida:** un dato puntual, el número que sale de medir un atributo una vez. La altura de una persona: 1,72 m. Los defectos encontrados en un módulo: 12.
- **Métrica:** relaciona medidas para expresar el grado en que algo posee un atributo. La altura promedio según el sexo, la edad o el país. Los defectos cada mil líneas de código.
- **Indicador:** una métrica, o una combinación de métricas, que da comprensión suficiente para tomar una decisión sobre el proceso, el proyecto o el producto (Pressman). Lo que lo distingue no es la fórmula sino que habilita una acción.

4.2 El ejemplo de la altura

El ejemplo que mejor fija los tres términos no menciona el software:



Figura 1. De un dato suelto a una decisión: medida, métrica, indicador y acción.

De aquí sale una regla: no todas las métricas son indicadores, pero todos los indicadores son métricas. Se recolectan medidas y se construyen métricas con el objetivo de obtener indicadores.

4.3 Indicadores clave (KPI)

En gestión se habla de KPI (*Key Performance Indicator*, indicador clave de rendimiento): los pocos indicadores que realmente muestran si la organización cumple los objetivos que se fijó, y que se agrupan en tableros de control. Un buen KPI es medible, cuantificable (tiene unidad), específico, temporal (se mide con una periodicidad definida) y relevante para la organización. Antes de definir uno conviene preguntarse qué se quiere medir y por qué, qué objetivo sigue, si es clave o sólo interesante, y quién lo supervisa y cada cuánto: las mismas preguntas que aparecen al construir una métrica propia (sección 8.4).

5. Tipos de medidas y escalas

5.1 Medidas directas e indirectas

Una medida es directa cuando se obtiene comparando con un patrón o contando: líneas de código, horas trabajadas, defectos encontrados. Es indirecta cuando se calcula a partir de otras: la productividad (líneas ÷ horas) o la densidad de defectos (defectos ÷ KLDC).

Casi todo lo que importa en software (calidad, productividad, mantenibilidad, complejidad) es indirecto: no se lee en un instrumento, hay que construirlo, y por eso puede estar mal construido. Los Puntos de Función de la Parte III son un ejemplo: se cuentan cosas observables para obtener una medida indirecta del tamaño.

5.2 Una pregunta incómoda

Tenemos tres defectos, con severidad baja, media y alta. ¿Cuál es la severidad promedio? La pregunta no tiene sentido: podemos ordenarlos, pero no sabemos si la distancia entre «baja» y «media» es la misma que entre «media» y «alta». Si les asignamos 1, 2 y 3, el promedio da 2; con 1, 2 y 10 daría 4,3. El número existe, pero no significa nada.

5.3 Las cinco escalas

No todos los números significan lo mismo. La escala en la que se mide un atributo decide qué operaciones tienen sentido sobre los valores. La teoría de la medición distingue cinco escalas:

Escala	Qué permite	Operaciones válidas	Ejemplo en software
Nominal	Sólo distinguir categorías	Contar por categoría, moda	Lenguaje de programación, tipo de defecto
Ordinal	Ordenar; las distancias no significan nada	Lo anterior + mediana, percentiles. No se promedia ni se multiplica	Severidad baja/media/alta, nivel de madurez CMMI
Intervalo	Sumar y restar; el cero es arbitrario	Lo anterior + promedio, diferencias	Fechas, temperatura en °C
Razón	Todo, incluida la división; el cero es real	Lo anterior + razones, porcentajes, multiplicación	LDC, horas, defectos, costo
Absoluta	Conteo directo de la propia entidad	Todas	Cantidad de módulos, cantidad de personas

5.4 Un error que ya evitamos

En la Unidad 2 multiplicamos probabilidad $\times$ impacto para obtener la exposición. Esa multiplicación es válida sólo porque antes fijamos el impacto en una escala numérica definida (de 1 a 4). Con *bajo* / *medio* / *alto* habría sido ordinal, y multiplicar habría sido inventar un número. El mismo cuidado reaparece en los Puntos de Función, con los pesos y los factores de ajuste.

IDEA CLAVE

Antes de operar con un número hay que preguntarse en qué escala está. Promediar o multiplicar valores ordinales produce resultados con apariencia de dato que no significan nada.

6. Qué se mide en software, y para qué

6.1 Proceso, producto y proyecto

En software hay tres entidades que se miden y que se confunden todo el tiempo:

Entidad	Qué es	Ejemplos de métricas	Qué responde
Proceso	Cómo desarrollamos	Defectos inyectados y removidos por fase, tiempo de ciclo, eficiencia de las pruebas	Cómo trabajar mejor en el próximo proyecto
Producto	El software en sí	Tamaño, complejidad, densidad de defectos, rendimiento	Cómo salió esto
Proyecto	La gestión del trabajo	Desvío de plazo, desvío de costo, uso de recursos	Si vamos bien ahora

Confundirlas lleva a usar un número para lo que no sirve: un atraso de dos semanas no dice nada de la calidad del producto, y la densidad de defectos de una versión no dice, sola, si el proceso es bueno. La distinción reaparece en la Unidad 4, al separar la calidad del producto de la del proceso.

6.2 Por qué medimos

La razón más directa está en la propia definición de la disciplina. El IEEE define la ingeniería de software como *la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software*. Sin medición, la palabra «ingeniería» queda vacía.

Park, Goethert y Florac (Software Engineering Institute) ordenan las razones para medir en cuatro: caracterizar (establecer una línea de referencia), evaluar el avance respecto de lo planificado, predecir entendiendo la relación entre proceso y producto, y mejorar identificando problemas y causas.

Dos frases clásicas completan el argumento. El principio de Gilb: *los proyectos sin metas claras no van a alcanzar sus metas claramente*, porque nadie sabe cuándo llegó. Y Lord Kelvin (1883): cuando se puede expresar en números aquello de lo que se habla, se sabe algo de eso; cuando no, el conocimiento es pobre e insatisfactorio. Es una exageración, pero recuerda que una afirmación sin número es difícil de verificar.

7. El proceso de medición

Nadie «mide y ya». La medición es un proceso con cinco actividades, y saltarse cualquiera de ellas la vuelve inútil:

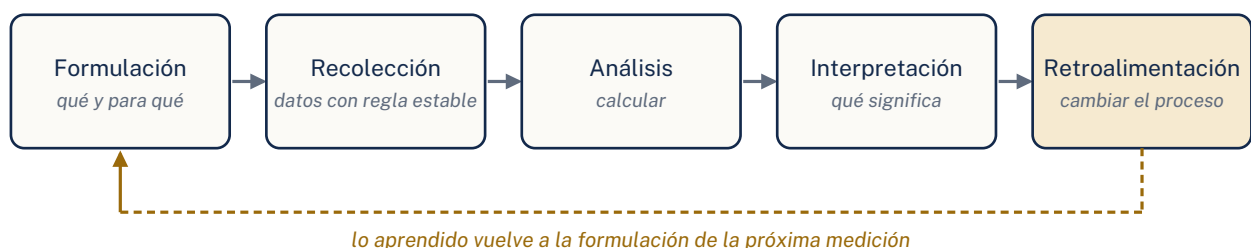


Figura 2. Las cinco actividades del proceso de medición.

- **Formulación:** qué se mide, para qué, y qué decisión se espera tomar. Si no se sabe qué decisión va a habilitar, conviene no medir.
- **Recolección:** juntar los datos con un procedimiento definido y estable; si cada uno cuenta distinto, no se puede comparar.
- **Análisis e interpretación:** calcular el número, y después decir qué significa en este contexto. Son dos pasos: el número no se explica solo.
- **Retroalimentación:** devolver lo aprendido al equipo y al proceso. Es el paso que más se saltea, y el que convierte la medición en mejora.

IDEA CLAVE

Medir sirve para convertir opiniones en algo que se pueda discutir, comparar y usar para decidir. Si un número no cambia ninguna decisión, no hacía falta medirlo.

8. Qué hace buena a una métrica

8.1 Una métrica mala, para empezar

Supongamos que una empresa anuncia: «*Vamos a medir la productividad de los programadores en líneas de código por programador por día*». Antes de seguir leyendo, piensen qué está mal.

Las objeciones salen solas: premia el código largo, castiga el refactor (quien simplifica un módulo tiene productividad negativa ese día), no distingue lenguajes, ignora la calidad y no mide el trabajo que no es código: diseñar, revisar, probar, hablar con el cliente.

8.2 Los criterios

Cada una de esas objeciones es la violación de un criterio. Una buena métrica tiene que ser:

Criterio	Qué significa	Cómo falla «LDC por programador por día»
Objetiva	Dos personas que la calculan sobre lo mismo obtienen el mismo resultado	¿Se cuentan comentarios, líneas en blanco, código generado? Sin regla, cada uno cuenta distinto
Simple	Se entiende qué mide y cómo se calcula	Ésta sí lo es; justamente por eso es tentadora
Obtenible	Se puede calcular con un costo razonable	También lo es: cualquier herramienta cuenta líneas
Válida	Mide el atributo que dice medir	Dice medir productividad y mide volumen de texto
Robusta	No se altera fácilmente ni se puede manipular sin cambiar lo que mide	Basta con escribir más líneas para «mejorar» el número

8.3 La validez de la medida

De los cinco criterios, el que el programa de la materia nombra explícitamente es la validez: una medida es válida si mide efectivamente el atributo que dice medir. Las líneas de código son una medida válida del *tamaño del texto* de un programa, no de la productividad ni de la funcionalidad. La métrica de la sección 8.1 es simple y fácil de obtener, y aun así es inútil porque no es válida para lo que se la quiere usar.

Una forma práctica de poner a prueba la validez es preguntarse: *¿puede mejorar el número sin que mejore el atributo?*, y al revés, *¿puede mejorar el atributo sin que se mueva el número?* Si cualquiera de las dos respuestas es sí, la métrica tiene un problema de validez. Para «LDC por día», las dos respuestas son sí.

8.4 Cómo construir una métrica

Como casi todo lo que importa es una medida indirecta, no alcanza con buscar métricas en un catálogo: hay que saber construirlas. Una receta en cuatro pasos:

- Identificar el atributo abstracto que se quiere medir. Por ejemplo, que un mozo pueda cargar un pedido rápido y sin equivocarse.
- Definir qué es «bueno» y «malo» para ese atributo: pocos pasos y pocos errores de carga, o muchos clics y correcciones frecuentes.
- Encontrar medidas observables, cosas que se puedan contar: tiempo de carga de un pedido, pedidos corregidos después de enviarlos, pedidos totales.
- Combinarlas en una métrica: tiempo promedio de carga y porcentaje de pedidos corregidos, por semana.

El ejercicio es hacer lo mismo con un atributo propio del sistema de cada grupo y someter la métrica a los cinco criterios. Si sobrevive, todavía falta la prueba de la sección siguiente.

9. Cómo se corrompen las métricas

9.1 ¿Y si el jefe la usa para los aumentos?

Supongan que construyeron una buena métrica para su proyecto. Ahora imaginen que, a partir del lunes, el jefe la usa para decidir los aumentos de sueldo. ¿Qué le pasa a la métrica?

La gente empieza a trabajar para el número en lugar de para lo que el número medía, y como ninguna métrica captura por completo su atributo, siempre hay maneras de mejorar el número sin mejorar el atributo. La métrica era válida mientras nadie tenía incentivos para manipularla; en cuanto se convierte en objetivo, pierde la validez. Es lo que se conoce como ley de Goodhart: cuando una medida se convierte en objetivo, deja de ser una buena medida.

Métrica	Cómo se corrompe cuando se vuelve objetivo
Velocity de Scrum	Si se exige que suba, los story points se inflan: la historia que valía 3 pasa a valer 5.
Bugs cerrados por tester	Premia reportar muchos defectos triviales, partir uno en varios, o reabrir y volver a cerrar.
Cobertura de pruebas al 100%	Se logra con pruebas que ejecutan todo y no verifican nada (sin una sola aserción).
LDC por programador	La métrica de la sección 8.1: se escribe más código, no mejor código.

9.2 Un contraejemplo: la densidad de defectos

No todas las métricas son malas. La densidad de defectos (defectos por KLDC o por PF) sirve para comparar versiones o productos de tamaño distinto y orientar dónde revisar más. Enseña además que una cantidad de defectos sola no dice nada: dividida por el tamaño, que actúa como

normalizador, se vuelve comparable. Pero también se corrompe si se la usa para evaluar personas: alcanza con dejar de registrar defectos.

9.3 Métricas privadas

De aquí sale una regla que plantea Pressman: las métricas se usan sobre el proceso y el producto, no sobre las personas. Algunos datos son privados por naturaleza (los defectos que cada uno encontró en su propio código) y sirven para que esa persona mejore. En cuanto las métricas se usan para premiar o castigar individuos, la gente deja de reportar datos reales y se pierde la confianza en todo el sistema de medición.

IDEA CLAVE

La validez de una métrica no es una propiedad fija: se pierde cuando la métrica se convierte en objetivo. Una métrica sirve mientras nadie la use para premiar o castigar a quienes generan los datos.

10. Medir el producto: tamaño, productividad y complejidad

10.1 Tamaño: líneas de código

La medida de tamaño más obvia es la cantidad de líneas de código (LDC), o sus miles (KLDC): directa, de escala de razón, fácil de obtener. Para que sea objetiva hay que fijar una regla de conteo: ¿se cuentan comentarios, líneas en blanco, código generado, bibliotecas de terceros? No hay respuesta universal; lo que importa es elegir una regla, escribirla y aplicarla siempre igual. Cuando en la ISFPP cuenten las LDC finales, la regla tiene que ser la misma que usaron para estimarlas.

El problema más grave es otro: las LDC sólo se pueden contar cuando el programa ya existe. Son la medida de tamaño más obvia y sirven para medir lo construido, calibrar y normalizar, pero son inútiles para lo que más se necesita al comenzar: estimar antes de escribir la primera línea. Ese es el problema con el que arranca la Parte II, y el que resuelven los Puntos de Función.

10.2 Densidad de defectos y normalización

Las LDC se usan sobre todo como normalizador: defectos por KLDC, costo por LDC. Con un dato hipotético agregado al caso de los dos equipos (20 KLDC para A y 5 KLDC para B), la densidad de defectos encontrados es de 17,1 defectos/KLDC para A y de 36,8 para B: la comparación cambia por completo, y todavía faltaría saber cuántos quedaron sin encontrar.

10.3 Métricas y productividad

La productividad es lo producido dividido por el esfuerzo: LDC por persona-mes o PF por persona-mes. Un persona-mes es el trabajo de una persona durante un mes, la unidad habitual de esfuerzo. Compararla es traicionero:

- Entre lenguajes. En LDC por persona-mes, un equipo que usa un lenguaje de bajo nivel parece más productivo, porque escribe más líneas para lo mismo: la métrica castiga a la herramienta más eficiente. En PF por persona-mes el problema se corrige en buena parte.

- Entre proyectos. La productividad depende del tamaño, la complejidad, la estabilidad de los requisitos y la experiencia del equipo.
- Entre personas. Es el caso de la sección 9: usada para evaluar gente, deja de medir productividad.

El uso legítimo es otro: la productividad histórica de un equipo es el dato que permite convertir tamaño en esfuerzo, que es lo que se hace en la Parte III.

10.4 Complejidad, cohesión y acoplamiento

Además del tamaño interesa la estructura interna, que determina cuánto cuesta entender, probar y modificar el producto. Son conceptos de IS2 que reaparecen como métricas:

- Complejidad ciclomática (McCabe, 1976): la cantidad de caminos independientes del grafo de flujo, $V(G) = E - N + 2$ (aristas menos nodos más dos; equivalentemente, decisiones más uno). En IS2 indicaba cuántos casos de prueba de caja blanca hacen falta; como métrica, un valor alto señala módulos difíciles de entender y probar.
- Cohesión y acoplamiento (GRASP en IS2): ahora se pueden medir, por ejemplo contando de cuántas clases depende una clase. Son la bisagra hacia la mantenibilidad de las Unidades 4 y 5.
- Métricas orientadas a objetos (Chidamber y Kemerer, 1994): profundidad del árbol de herencia (DIT), número de hijos (NOC) y acoplamiento entre clases (CBO). Valores altos indican clases más difíciles de modificar sin efectos colaterales.

PARTE II

Estimar

El problema de la estimación · estimar y predecir · calidad de una estimación · métodos y modelos · COCOMO

11. El problema de la estimación

11.1 Las tres preguntas del primer día

Apenas arranca un proyecto alguien pregunta ¿cuánto va a durar?, ¿cuánto esfuerzo requiere? y ¿cuánta gente hace falta? El problema es cuándo se pregunta: en el momento de mayor ignorancia de todo el proyecto, sin diseño, sin código y a veces sin requisitos claros. Ese problema no se resuelve; se administra.

11.2 Qué hay que tener antes de estimar

No se puede estimar lo que no está definido. Tiene que estar claro el ámbito: la función, el rendimiento, las restricciones, las interfaces y la fiabilidad requerida. Si alguna dimensión está en blanco, la estimación va a incluir ese hueco, y el hueco tiene costo.

11.3 ¿Cuántos autos hay en el puente?

Imaginen un puente lleno de autos, en dos carriles. Nadie los va a contar de a uno. ¿Se puede saber cuántos hay?

Sí, midiendo otra cosa: el largo del puente, $Z = 100$ m; el largo promedio de un auto, $X = 4$ m; la separación entre autos, $Y = 0,5$ m. Cada auto ocupa $X + Y = 4,5$ m, así que entran $C = Z \div (X + Y) = 100 \div 4,5 \approx 22$ autos por carril.

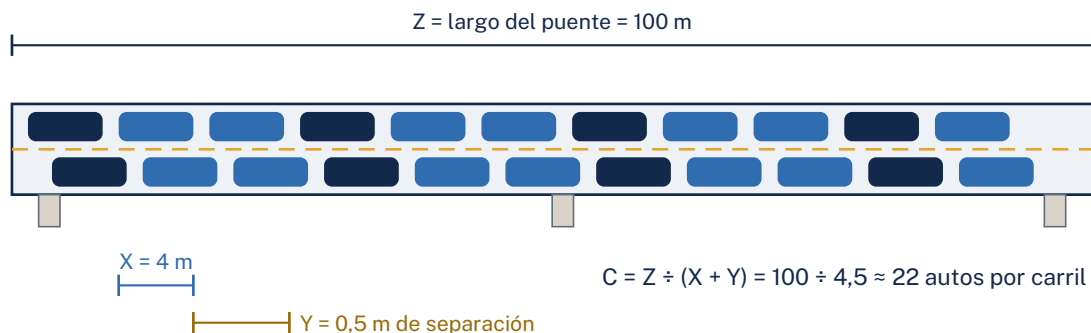


Figura 3. Estimar la cantidad de autos midiendo el puente y los autos, sin contarlos.

No se contó ningún auto. Eso es estimar: medir atributos internos que sí se pueden medir, para derivar uno que no se puede medir directamente.

11.4 Predecir es otra cosa

Otra forma de responder: contar los autos todos los sábados anteriores (22, 25, 23, 28, 20) y decir «el próximo sábado habrá unos 22 o 24». No se midió nada del puente: se usó la historia. Eso es predecir.

	Estimar	Predecir
Se apoya en	Atributos del propio objeto	Datos históricos de casos parecidos
Sirve cuando	No hay historia disponible	Hay historia confiable y comparable
En software	Puntos de Función	Analogía con proyectos previos; velocity en Scrum

En la industria se dice «estimo» para las dos cosas, pero cambia qué información hace falta: para estimar hay que poder medir el objeto; para predecir, tener datos de casos anteriores. Y hay un cierre que conviene adelantar: quien estima mal la separación entre autos estima mejor la segunda vez, siempre que registre el error. Esa idea se llama calibración, y reaparece en los modelos de estimación y en COCOMO.

12. Qué influye en el costo y en la productividad

Lo que hace caro a un software es la complejidad del problema, el tamaño del producto, el tiempo disponible (comprimir el plazo encarece, y estirarlo demasiado también), la confiabilidad requerida y el nivel tecnológico exigido.

El costo crece más que proporcionalmente con el tamaño: un sistema del doble de grande cuesta bastante más del doble, porque más partes significan más interacciones, más comunicación y más integración. Esta no linealidad aparece en los modelos como un exponente mayor que 1, y en la Parte III como la ley de Brooks.

La productividad del equipo depende de factores humanos (tamaño y experiencia), del problema (complejidad y cambios de requisitos), del proceso (técnicas, lenguajes, herramientas, revisiones), del producto (fiabilidad y rendimiento exigidos) y del recurso disponible.

13. La calidad de una estimación

13.1 De qué depende que una estimación valga algo

La calidad de una estimación no depende de la fórmula sino de cinco cosas: la estimación del tamaño o la complejidad, la capacidad de traducirlo en esfuerzo, cuánto el proyecto representa las habilidades reales del equipo, la estabilidad de los requisitos y la estabilidad del entorno.

Las dos primeras son cuestiones técnicas: se pueden mejorar estudiando, contando mejor y calibrando con datos propios. Las otras tres no. Si los requisitos se mueven o el equipo cambia, la estimación se vence, y no hay método que lo arregle. Por eso una estimación siempre tiene que ir acompañada de los supuestos sobre los que se hizo: si cambian los supuestos, hay que volver a estimar.

13.2 El cono de la incertidumbre

Al principio de un proyecto, una estimación puede errar por un factor de 4 para arriba o para abajo, y el rango se angosta a medida que se avanza. Boehm lo documentó con datos de proyectos reales y McConnell lo popularizó como cono de la incertidumbre.

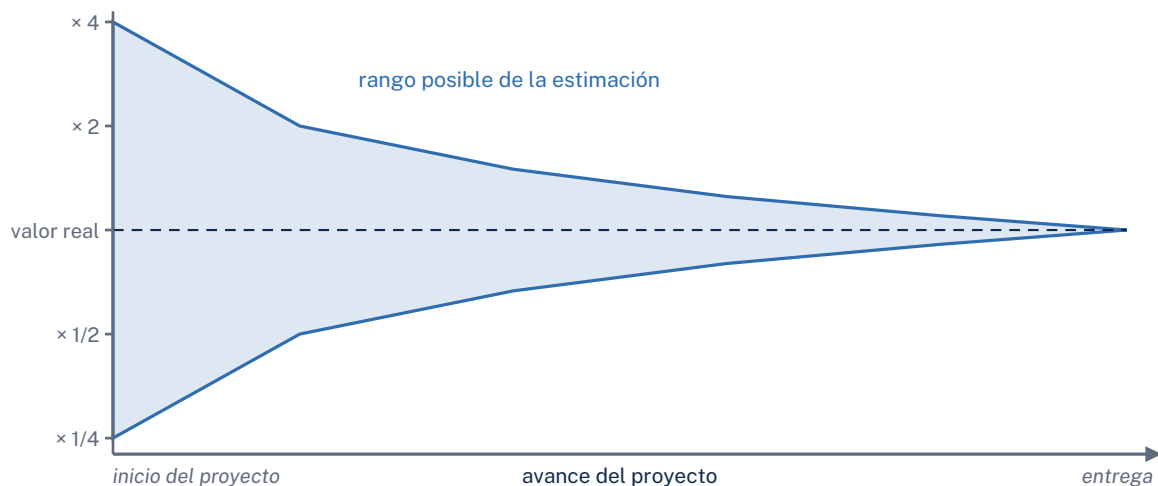


Figura 4. El cono de la incertidumbre: el error posible de una estimación se reduce a medida que avanza el proyecto (escala logarítmica).

El rango se angosta no porque se estime mejor, sino porque se sabe más. Por eso la estimación tiene que ser un proceso continuo, con refinamientos sucesivos: una estimación del primer día que nunca se revisa es un número viejo con aspecto de dato.

13.3 Estimar no es comprometerse

Estimar es calcular cuánto podría costar algo; comprometerse es prometer una fecha y un precio. Aunque los decida la misma persona en la misma reunión, son dos decisiones distintas:

- La estimación es un rango con probabilidades; el compromiso es un punto de ese rango, con la probabilidad de fallar que trae.
- La estimación responde «¿cuánto cuesta?»; el compromiso, «¿cuánto riesgo asumo?».
- La estimación se corrige cuando se sabe más; el compromiso no.

El meteorólogo estima un 70% de probabilidad de lluvia; no se compromete a que llueva. Si se le exigiera comprometerse, no se obtendría mejor información sino un pronóstico complaciente.

La ventaja práctica de separarlas es que el margen queda a la vista. Si se estima 6 meses y se promete 8, esos 2 meses son el margen: se lo puede defender, ajustar o cambiar por alcance. Si se «estima» 8 de entrada, el colchón queda escondido en el número, y en la próxima negociación se lo recortan sin saber qué se recorta. Y si el cliente quiere escuchar 4 y se le dice 4, ya no es un cálculo: es una negociación presentada como análisis.

Por eso una estimación profesional se entrega como un rango, más ancho cuanto más incertidumbre haya: un número exacto cuando no se sabe esconde la ignorancia.

14. Los métodos de estimación

Hay cuatro familias de métodos: la analogía con proyectos parecidos, el juicio de expertos, la descomposición en subproblemas más simples y los modelos de estimación, que son fórmulas que relacionan parámetros del proyecto con el esfuerzo y el costo.

14.1 Estimación por analogía

Se estima a partir de proyectos comparables ya terminados, con datos cuantificados: es predicción. Tiene dos problemas: el tamaño no escala lineal, y el entorno tiene que ser comparable (mismo equipo, mismas herramientas, tamaño y complejidad parecidos).

«Del mismo rubro» no alcanza. Dos sistemas para una ferretería: una caja registradora con cinco pantallas, y otro con depósitos múltiples, compras, facturación electrónica y tienda web. Mismo dominio y, fácilmente, veinte veces el trabajo. Lo que tiene que ser comparable es el tamaño y la complejidad, y para saberlo hay que haberlos medido. La analogía sin datos medidos se reduce a «me parece que se parece»: el método más usado depende de la Parte I de esta unidad.

14.2 Juicio de expertos

Se estima con la opinión de personas con experiencia en el dominio y en sistemas de complejidad parecida. El problema conocido es que es subjetivo, y que en una reunión el primero que dice un número arrastra a los demás. Es el anclaje que vieron entre los sesgos cognitivos de la Unidad 2, ahora aplicado a la estimación.

Contra eso existe el método Delphi, nacido en la RAND Corporation: un coordinador reparte la especificación; cada experto estima en forma anónima; el coordinador comparte el resumen (por ejemplo, el valor medio) y pide otra ronda, hasta llegar a un consenso razonable.

La variante Delphi banda ancha (*wideband Delphi*), que Boehm describe en 1981, agrega reuniones donde los expertos discuten las diferencias: después de cada ronda anónima, el grupo discute por qué difieren las estimaciones (en especial las más alejadas) y se vuelve a estimar hasta converger. La discusión hace aflorar supuestos que alguien tenía y los demás no.

En la literatura el juicio de expertos suele describirse como un ejercicio de adivinación; sin embargo es el método más usado en la industria, y los estudios no muestran que sus resultados sean sistemáticamente peores que los de métodos en apariencia más precisos.

14.3 Descomposición en subproblemas

Se parte el sistema en piezas chicas, se estima cada una y se suman las estimaciones. Funciona porque estimamos mucho mejor lo chico que lo grande: el error de cada parte es menor, y los errores de las distintas partes tienden a compensarse entre sí.

EJERCICIO

Tomen tres funciones de su ISFPP y estimen, por descomposición, cuántas horas lleva cada una. Sumen. ¿El total les parece razonable? Casi siempre la suma da bastante menos de lo que después lleva en realidad. Ahí aparecen el sesgo de optimismo de la Unidad 2 y todo lo que no se contó: pruebas, integración, reuniones, correcciones.

14.4 Modelos de estimación

La cuarta familia son las fórmulas empíricas que relacionan el tamaño con el esfuerzo. Su forma general es:

$$E = a + b \times (ve)^c$$

donde E es el esfuerzo (en personas-mes), ve es la variable de estimación (LDC o Puntos de Función) y a, b, c son coeficientes que se obtuvieron ajustando la fórmula a datos de proyectos reales. Cuando c es mayor que 1, la fórmula refleja la no linealidad de la sección 12: el esfuerzo crece más rápido que el tamaño. La bibliografía (Pressman, entre otros) recopila varios modelos de este tipo:

Modelo	Fórmula	Variable
Walston-Felix	$E = 5,2 \times (KLDC)^{0,91}$	KLDC
Bailey-Basili	$E = 5,5 + 0,73 \times (KLDC)^{1,16}$	KLDC
Boehm (COCOMO)	$E = 3,2 \times (KLDC)^{1,05}$	KLDC
Doty (para KLDC > 9)	$E = 5,288 \times (KLDC)^{1,047}$	KLDC
Albrecht y Gaffney	$E = -13,39 + 0,054 \times PF$	PF
Kemerer	$E = 60,62 \times 7,728 \times 10^{-8} \times PF^3$	PF
Matson-Barnett	$E = 585,7 + 15,12 \times PF$	PF

14.5 El mismo sistema en todos los modelos

Tomemos un sistema de 25.000 líneas en Cobol, que equivalen a unos 250 Puntos de Función, y metamos ese mismo dato en cada fórmula:

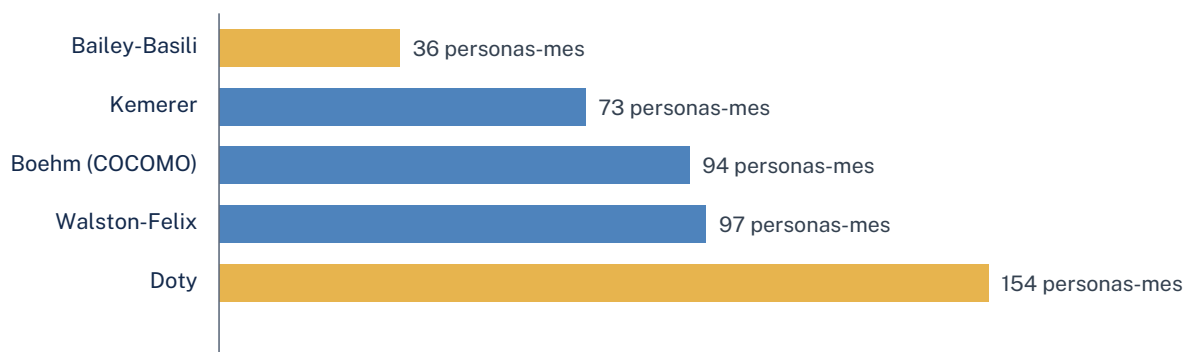


Figura 5. Esfuerzo estimado para el mismo sistema (25 KLDC / 250 PF) con cinco modelos empíricos.

El más bajo da 36 personas-mes; el más alto, 154. Más de cuatro veces de diferencia, con el mismo sistema, el mismo tamaño y los mismos datos de entrada, y ninguna cuenta está mal

hecha. Los dos modelos restantes de la tabla dan resultados directamente inservibles: Albrecht y Gaffney da 0,11 personas-mes (prácticamente cero) y Matson-Barnett da 4.366 personas-mes (más de trescientos años-persona).

Cada modelo se ajustó con los proyectos de una organización distinta: otro tipo de sistema, otro equipo, otras herramientas, otra época. Los coeficientes no son constantes de la naturaleza. Un modelo sin calibrar es una fórmula ajena; para que sirva hay que recalcular sus coeficientes con proyectos propios terminados, y para eso hay que haber medido tamaño y esfuerzo real. Sin medición no hay calibración.

La dispersión también es información: si cinco modelos dan entre 36 y 154, eso mide la incertidumbre. Un rango ancho significa que todavía no se sabe lo suficiente, y eso también sirve para decidir.

14.6 Usar varios métodos a la vez

No se elige un método: se usan varios, cada uno como comprobación de los otros. Si dan parecido, hay base para confiar; si dan muy distinto, entender por qué suele revelar un supuesto oculto.

15. COCOMO

El modelo empírico más conocido es COCOMO (*CO*nstructive *CO*st *MO*del), que Barry Boehm publicó en 1981 en *Software Engineering Economics* a partir de datos de 63 proyectos. Se presenta en tres niveles de detalle: básico, intermedio y detallado. Aquí vemos los dos primeros.

15.1 COCOMO básico

COCOMO básico estima el esfuerzo sólo a partir de las KLDC, con coeficientes que dependen del modo del proyecto: orgánico (equipos chicos con experiencia, requisitos poco rígidos, entorno estable), semiacoplado (situación intermedia) o empotrado (restricciones fuertes de hardware, software u operación).

Las ecuaciones son $E = a \times (KLDC)^b$ para el esfuerzo en personas-mes y $T = c \times E^d$ para el plazo de desarrollo en meses. La cantidad media de personas surge de dividir E por T.

Modo	a	b	c	d
Orgánico	2,4	1,05	2,5	0,38
Semiacoplado	3,0	1,12	2,5	0,35
Empotrado	3,6	1,20	2,5	0,32

El exponente b es siempre mayor que 1 y crece del modo orgánico al empotrado: es la no linealidad del tamaño. El exponente d es bastante menor que 1: el plazo crece mucho más despacio que el esfuerzo, porque no se puede repartir el esfuerzo entre cualquier cantidad de gente.

15.2 COCOMO intermedio

COCOMO intermedio agrega al tamaño quince conductores de costo (*cost drivers*) agrupados en atributos del producto (por ejemplo, fiabilidad requerida y complejidad), de la plataforma (restricciones de tiempo y memoria), del personal (capacidad y experiencia de analistas y programadores) y del proyecto (uso de herramientas, plazo exigido). Cada uno se califica de «muy

bajo» a «extra alto», y cada calificación tiene un multiplicador. El producto de los quince es el factor de ajuste del esfuerzo (EAF), y la ecuación pasa a ser $E = a \times (KLDC)^b \times EAF$, con $a = 3,2$ (orgánico), $3,0$ (semiacoplado) o $2,8$ (empotrado) y los mismos exponentes b del modelo básico.

Con todos los conductores en su valor nominal, $EAF = 1$ y la fórmula del modo orgánico queda $E = 3,2 \times (KLDC)^{1,05}$: la fila «Boehm (COCOMO)» de la sección 14.4. Los conductores cumplen el mismo papel que los factores de ajuste de los Puntos de Función.

15.3 Para qué sirve, y para qué no

COCOMO es explícito: las fórmulas están publicadas, cualquiera puede repetir la cuenta, y obliga a pensar en factores que la intuición olvida. Pero sus coeficientes provienen de proyectos de fines de los setenta; usado tal cual es una fórmula ajena, y su valor aparece cuando se lo calibra con proyectos propios. En 2000 se publicó COCOMO II, pensado para procesos y lenguajes más modernos, que admite como entrada LDC o Puntos de Función.

Además, COCOMO necesita las KLDC, que al comienzo no se conocen. La manera más defendible de estimarlas es pasar por los Puntos de Función.

IDEA CLAVE

Estimar no es adivinar ni es una ciencia exacta. Es medir lo que se puede medir, usar la experiencia registrada, cruzar métodos, entregar un rango honesto y volver a estimar cuando se sepa más.

PARTE III

Puntos de Función

Medir la funcionalidad para estimar el esfuerzo · el sistema de fútbol de punta a punta · estimación en equipos ágiles

16. La idea de los Puntos de Función

16.1 La pregunta imposible y la pregunta posible

¿Cuántas líneas de código va a tener su ISFPP? Nadie lo sabe. ¿Cuántas pantallas de carga, cuántos reportes, cuántas entidades? Eso sí se puede contestar hoy, con el enunciado en la mano. Si no se puede medir el código, se mide lo que el sistema hace para el usuario y de ahí se deriva el esfuerzo: la misma jugada del puente.

16.2 Albrecht y el tamaño funcional

Los Puntos de Función (PF) fueron propuestos por Allan Albrecht, de IBM, en 1979, como medida del tamaño funcional: se cuenta lo que el sistema le ofrece al usuario, no cómo está construido. Hoy el método lo mantiene el IFPUG (*International Function Point Users Group*); acá se presenta la versión clásica simplificada. Se puede calcular en la etapa de análisis, es independiente del lenguaje y permite comparar proyectos hechos con herramientas distintas: tres cosas que las LDC no cumplen.

17. El pedido del club

Para aprender a contar usamos un sistema chico, cuyo pedido llega como los reales: en prosa, contado por el cliente.

EL PEDIDO DEL CLUB

Tenemos una liga de fútbol amateur acá en Trelew, con equipos de los barrios. Hoy manejamos todo en papel y ya no da más. Necesitamos poder cargar los equipos que participan y también ir cargando los partidos que se juegan: quién jugó contra quién, la fecha y el resultado. Los datos de los equipos cambian seguido (cambia el delegado, cambia la cancha donde hacen de local), así que tenemos que poder corregirlos. Y cuando un equipo se baja de la liga, sacarlo del sistema. Lo que más nos piden es consultar. La gente llama para saber cuándo juega tal equipo, así que necesitamos poder buscar un partido por fecha. Y para las reuniones de comisión necesitamos sacar el listado de todos los equipos, un reporte de los equipos que se anotaron en un rango de fechas, y un reporte de los partidos jugados.

Antes de seguir, hagan la lista de funciones y de cosas que hay que guardar, y compárenla con ésta:

#	Función	#	Función
1	Registrar equipos	5	Buscar partido por fecha
2	Registrar partidos	6	Listado de equipos
3	Actualizar datos de equipos	7	Reporte de equipos por rango de fechas
4	Eliminar equipos	8	Reporte de partidos

Las entidades son cuatro: equipo, partido, delegado y cancha. Las dos últimas casi nunca aparecen en la primera lista, porque están en una frase subordinada; sin embargo valen 20 de los 75 puntos del sistema. Lo que no se identificó no se cuenta, y lo que no se cuenta no se estima.

18. Los cinco componentes

18.1 Funciones de transacción y funciones de datos

Las funciones de la lista no son todas del mismo tipo, y las entidades son otra cosa. Todo sistema procesa y guarda: hay funciones de transacción (lo que el usuario hace: ingresa datos, pide informes, consulta) y funciones de datos (lo que el sistema guarda o consulta de otros sistemas). En total son cinco componentes, con las siglas del estándar IFPUG:

Sigla	Nombre	Qué es
EI	<i>External Input</i>	El usuario ingresa datos que modifican lo que el sistema guarda: altas, bajas, modificaciones.
EO	<i>External Output</i>	El sistema entrega información elaborada: informes, listados con totales, gráficos. Hay algún cálculo o derivación.
EQ	<i>External Inquiry</i>	El usuario pide y ve datos, sin cálculo y sin modificar nada.
ILF	<i>Internal Logical File</i>	Datos que el sistema mantiene. En la práctica, casi siempre una entidad del modelo de dominio.
EIF	<i>External Interface File</i>	Datos que el sistema usa pero no mantiene: los administra otro sistema.

18.2 Las distinciones que más se confunden

EO o EQ. Las dos recuperan datos y los muestran. La diferencia es que la EO además procesa: totaliza, calcula, ordena según un criterio derivado. «Buscar el partido del 12 de marzo» es una EQ; «reporte de goles por equipo del último semestre» es una EO. La regla práctica: si hay una cuenta en el medio, es EO.

ILF o EIF. La pregunta es quién mantiene los datos. El sistema de fútbol no tiene ningún EIF: todos los datos que usa los carga y los mantiene él mismo. Si la liga trajera los datos de los jugadores desde un sistema de la federación, que el club consulta pero no modifica, eso sí sería un EIF.

«Archivo lógico» no es un archivo. ILF y EIF se cuentan como grupos de datos que el usuario reconoce, no según el diseño de la base: una entidad con sus tablas auxiliares puede ser un solo ILF.

18.3 Ojo con la bibliografía

En los libros en castellano las siglas aparecen traducidas: EE (Entrada Externa) por EI, SE (Salida Externa) por EO, CE (Consulta Externa) por EQ, ALI (Archivo Lógico Interno) por ILF y AIE (Archivo de Interfaz Externo) por EIF. Son las mismas cosas con otro nombre; en la materia usamos las siglas inglesas y no las mezclamos con nombres castellanos.

18.4 Cómo se reconocen en la práctica

Una guía para empezar es mirar el verbo: *registrar, actualizar, eliminar* suelen ser EI; *listar, informe, reporte*, EO; *buscar, consultar, ver*, EQ. Es una guía, no una regla: un «listado» sin ningún cálculo se parece más a una EQ, y no toda tabla es un ILF. Hay que mirar qué hace la función para el usuario, y ante un caso discutible, decidir un criterio, escribirlo y aplicarlo igual en todo el conteo.

19. Los pesos y el conteo

19.1 No todo pesa igual

Cada componente se pondera según su tipo y su complejidad. La tabla de pesos se fija antes de empezar a contar:

Componente	Simple	Medio	Complejo
EI · <i>External Input</i>	3	4	6
EO · <i>External Output</i>	4	5	7
EQ · <i>External Inquiry</i>	3	4	6
ILF · <i>Internal Logical File</i>	7	10	15
EIF · <i>External Interface File</i>	5	7	10

Las funciones de datos pesan mucho más: cada entidad implica estructuras, validaciones y lógica usadas desde muchas funciones. El IFPUG determina la complejidad de cada componente con tablas de datos elementales y archivos referenciados; para un primer conteo se toman todas de complejidad media.

19.2 El conteo del sistema de fútbol

Con el sistema ya conocido, el procedimiento es: clasificar primero, pesar después, sumar al final.

Función	Tipo	Cuenta × peso	PF
Registrar equipos	EI	1 × 4	4
Registrar partidos	EI	1 × 4	4
Actualizar datos de equipos	EI	1 × 4	4
Eliminar equipos	EI	1 × 4	4
Buscar partido por fecha	EQ	1 × 4	4
Listado de equipos	EO	1 × 5	5
Reporte de equipos por rango de fechas	EO	1 × 5	5
Reporte de partidos	EO	1 × 5	5
Entidades: equipo, partido, delegado, cancha	ILF	4 × 10	40
Total: Puntos de Función sin ajustar (PFSA)			75

El «listado de equipos» se contó como EO, aunque es discutible: si sólo muestra datos sin cálculo sería una EQ y el total bajaría un punto. Por eso el criterio de conteo se fija y se escribe antes de empezar.

El resultado es PFSA = 75, sin haber escrito una línea de código. Delegado y cancha aportan 20 de esos puntos. La misma medida aparece en la bibliografía y en el enunciado de la ISFPP como PFNA (No Ajustados); en este apunte usamos PFSA.

EJERCICIO

En la práctica van a contar los PFSA de otro sistema chico, con el enunciado a la vista. El orden importa: clasifiquen todas las funciones, después asignen los pesos y recién al final sumen.

20. Los factores de ajuste

20.1 Dos sistemas con los mismos PF

Dos sistemas cuentan 75 PF cada uno. Uno corre en una notebook, para diez usuarios internos. El otro es distribuido, funciona en seis sedes, maneja datos sensibles y tiene que responder en tiempo real. ¿Cuestan lo mismo? Evidentemente no. Lo que encarece al segundo no es *qué hace* (la funcionalidad es la misma) sino en qué condiciones tiene que hacerlo. Albrecht listó 14 características generales del sistema que capturan esas condiciones; la lista completa aparece en la tabla de la sección 20.3.

20.2 Cómo se puntúa

Cada factor recibe un puntaje de 0 a 5 según cuánta influencia tiene en este sistema concreto: 0 = ninguna, 1 = incidental, 2 = moderada, 3 = media, 4 = significativa, 5 = esencial. La suma de los catorce puntajes, ΣFi , queda entre 0 y 70. Lo que importa de cada puntaje no es el número sino que se pueda justificar.

Con esa suma se calcula el factor de ajuste y los Puntos de Función ajustados:

$$PFA = PFSA \times [0,65 + (0,01 \times \Sigma Fi)]$$

20.3 El ajuste del sistema de fútbol

Para el sistema de fútbol se toma $\Sigma Fi = 32$, que corresponde a un sistema sencillo, sin distribución ni requisitos especiales de rendimiento. La tabla siguiente muestra una puntuación posible que da esa suma, con su justificación; otra persona podría justificar valores algo distintos, y eso es parte del método.

#	Factor	Fi	Justificación
1	Comunicación de datos	3	Varias computadoras de la liga, en red
2	Procesamiento distribuido	0	Todo se procesa en un solo lugar
3	Rendimiento	1	Sin exigencias especiales de respuesta
4	Uso intensivo de la configuración	1	Equipos comunes, sin restricciones
5	Tasa de transacciones	1	Pocas operaciones por día
6	Entrada de datos en línea	5	Toda la carga es interactiva
7	Eficiencia para el usuario final	4	Consultas telefónicas: la búsqueda debe ser ágil
8	Actualización en línea	4	Los datos se corrigen en el momento
9	Procesamiento complejo	1	Casi sin cálculos; totales en reportes
10	Reusabilidad	2	No se pide reutilizar componentes
11	Facilidad de instalación	3	Lo instala la liga, sin soporte técnico
12	Facilidad de operación	4	Lo operan voluntarios sin formación técnica
13	Múltiples sitios	0	Un solo lugar de uso

#	Factor	Fi	Justificación
14	Facilidad de cambio	3	Es esperable que pidan cambios en reportes
	ΣFi	32	

Con PFSA = 75 y $\Sigma Fi = 32$:

$$PFA = 75 \times [0,65 + (0,01 \times 32)] = 75 \times [0,65 + 0,32] = 75 \times 0,97 = 72,75 \approx 73 \text{ PF}$$

20.4 El ajuste mueve poco

Con $\Sigma Fi = 0$ el factor vale 0,65; con 35, 1,00; con 70, 1,35. El ajuste sólo mueve el resultado un 35% para arriba o para abajo: lo que define el tamaño es el conteo. Conviene invertir el tiempo en contar bien y no en discutir si un factor va en 3 o en 4, que mueve el resultado apenas un 1% de los PFSA.

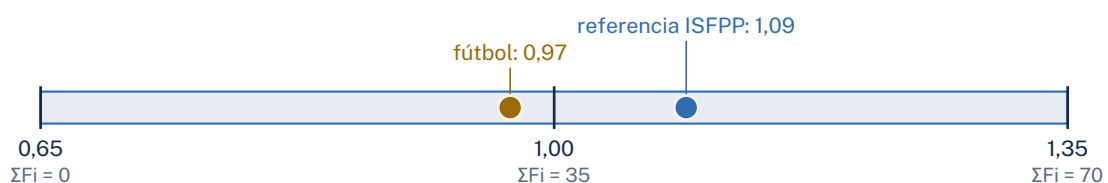


Figura 6. Rango posible del factor de ajuste, con el valor del sistema de fútbol y la referencia que da el enunciado de la ISFPP.

21. De los Puntos de Función al costo

Los PF miden cuán grande es el sistema. Todavía no dicen cuántos meses ni cuánto dinero va a costar. Para eso hay que convertir, y en cada conversión entran datos históricos.



cada flecha agrega incertidumbre

Figura 7. La cadena completa de la estimación por Puntos de Función.

21.1 De PF a líneas de código

Cada lenguaje necesita más o menos código para implementar la misma funcionalidad. Las tablas de la industria (Capers Jones, QSM) dan órdenes de magnitud:

Lenguaje	LDC por PF	Lenguaje	LDC por PF
Assembler	~320	Python	~30
C	~128	SQL	~15
Java / C#	~55	Generadores de código	~15
JavaScript / TypeScript	~47		

Son promedios aproximados: los valores propios de la organización siempre son mejores que los de la tabla.

Un sistema moderno rara vez usa un solo lenguaje. La ISFPP de este año se hace con React + FastAPI + SQLite (TypeScript, Python y SQL), así que hay que repartir la funcionalidad por capa y convertir cada parte con su factor. El reparto se justifica mirando dónde vive la funcionalidad, y se corrige al final con las LDC reales.

Si el sistema de fútbol se construyera con ese mismo stack, y con un reparto de partida razonable, la conversión de sus 73 PF sería:

Capa	Lenguaje	Reparto	PF	LDC/PF	LDC
Frontend	React / TypeScript	45%	32,85	47	1.544
Backend	Python / FastAPI	45%	32,85	30	986
Base de datos	SQL	10%	7,30	15	110
Total		100%	73		2.639

Es decir, unas 2.640 LDC (2,64 KLDC). Los porcentajes son un punto de partida, no una regla: cada grupo tiene que estimar el suyo según dónde viva la lógica de su sistema.

21.2 De PF a esfuerzo

Hay dos vías. La vía corta usa la productividad histórica del equipo, medida en PF por persona-mes (sección 10.3). Si el equipo produce, por ejemplo, 6,5 PF por persona-mes:

$$E = 73 \text{ PF} \div 6,5 \text{ PF/persona-mes} \approx 11,2 \text{ personas-mes}$$

La otra vía pasa por las líneas de código: PF → LDC → COCOMO → esfuerzo. Sirve cuando hay datos históricos de LDC pero no de PF. Con COCOMO básico en modo orgánico (un equipo chico, con requisitos poco rígidos) y las 2,64 KLDC de la sección anterior:

$$E = 2,4 \times (2,64)^{1,05} \approx 6,6 \text{ personas-mes}$$

Las dos vías no dan lo mismo. COCOMO se calibró con lenguajes que necesitaban muchas más líneas por PF que TypeScript o Python, así que con pocas LDC tiende a dar poco esfuerzo; y la productividad de 6,5 PF por persona-mes es un valor de ejemplo, no un dato de nuestro equipo. Lo correcto es informar el rango, entre 6,6 y 11,2 personas-mes, dejando escrito de dónde sale cada extremo.

21.3 De esfuerzo a plazo

El esfuerzo no se divide por la gente disponible. Un sistema de 372 PF, con la misma productividad, lleva unas 57 personas-mes, y eso no significa que 57 personas lo terminen en un mes. Es la ley de Brooks (Unidad 1): agregar gente a un proyecto atrasado lo atrasa más, porque hay que capacitarla y se multiplican los canales de comunicación.

La ecuación de plazo de COCOMO orgánico, $T = 2,5 \times E^{0,38}$, da para ese ejemplo unos 11,6 meses con un equipo medio de 5 personas. Para el sistema de fútbol, el plazo va de 5,1 a 6,3 meses, con 1,3 a 1,8 personas en promedio.

21.4 De esfuerzo a costo

Costo = esfuerzo (personas-mes) × costo mensual cargado de una persona. «Cargado» incluye, además del sueldo, cargas sociales, infraestructura y licencias, y la parte proporcional del tiempo que no se factura.

Con un costo mensual cargado hipotético de USD 3.000 (sólo para hacer la cuenta, no un dato de mercado), el sistema de fútbol costaría entre USD 19.945 y USD 33.692.

21.5 El resultado completo

Paso	Sistema de fútbol	De dónde sale
PFSA	75	Conteo de 8 funciones y 4 entidades, complejidad media
PFA	$72,75 \approx 73$	$\Sigma Fi = 32$, factor 0,97
LDC	≈ 2.640	Reparto por capa 45 / 45 / 10 y factores 47 / 30 / 15
Esfuerzo	6,6 a 11,2 personas-mes	COCOMO básico orgánico; productividad de 6,5 PF/persona-mes
Plazo	5,1 a 6,3 meses	$T = 2,5 \times E^{0,38}$
Costo	USD 19.945 a 33.692	Costo mensual cargado hipotético de USD 3.000

Cada flecha agrega incertidumbre (el conteo, el reparto por capa, los factores de conversión, la productividad), por eso se entrega un rango explicando cada paso. Y la estimación es este rango; el compromiso es otra decisión, tomada aparte y con el margen a la vista (sección 13.3).

22. Lo que los Puntos de Función no hacen

Los PF no miden calidad (300 PF pueden estar bien o mal contruidos), no miden dificultad algorítmica (un motor de cálculo pesa poco) y no sirven para comparar personas (sección 9).

Tienen además críticas legítimas: cuesta trabajo contarlos; el conteo depende de quién cuenta, por lo que hay que fijar criterios (un problema de objetividad, sección 8.2); y pierden precisión en sistemas de mucho cálculo y poca interacción. Aun así, se pueden calcular en la etapa de análisis, cosa que ninguna otra medida de tamaño permite.

23. Equivocarse es parte del método

Ninguna estimación es exacta. Lo que la vuelve útil es que quede escrita, con sus supuestos, y que después se compare contra lo que pasó. Esa comparación no vale por haber acertado sino por poder explicar la diferencia: qué supuesto falló, qué se contó mal, qué no se contó.

Una organización que estima y nunca revisa no mejora. Una que registra sus errores calibra sus factores de conversión y su productividad, y en dos o tres proyectos estima bastante bien.

IDEA CLAVE

No se puede medir el código que todavía no se escribió, pero sí se puede contar lo que el sistema va a hacer. Con ese conteo y con los datos de lo que pasó en proyectos anteriores, se obtiene una estimación que se puede defender.

24. Estimar en equipos ágiles

24.1 Una pregunta incómoda

Casi nadie que trabaja en desarrollo usa Puntos de Función. No es que estén mal: PF y las técnicas ágiles resuelven momentos distintos del mismo problema, y pertenecen a la misma familia de ideas.

24.2 Story points

En Scrum, las historias de usuario no se suelen estimar en horas sino en puntos de historia (*story points*). Un punto no es una unidad de tiempo: es una medida de tamaño relativo, que combina volumen de trabajo, complejidad e incertidumbre.

Somos malos estimando en absoluto y buenos comparando: cuesta decir cuántos kilos pesa un mueble, pero es fácil decir si pesa más que otro. Por eso se elige una historia de referencia y las demás se estiman en comparación con ella.

24.3 Planning poker

Así se estima una historia en equipo con planning poker:

- Se lee la historia y se aclaran las dudas.
- Cada integrante elige su carta en privado, sin decir nada.
- Se revelan todas las cartas a la vez.
- Quienes eligieron los valores extremos, el más alto y el más bajo, explican por qué.
- Se vuelve a votar hasta converger.

Estimación individual y secreta, revelación simultánea, discusión de las diferencias, nueva ronda: es Delphi banda ancha (sección 14.2). Planning poker es su versión con cartas, propuesta por James Grenning en 2002 y difundida después por Mike Cohn.

La misma idea apareció tres veces en la materia: en la Unidad 2 como antídoto contra el anclaje, en esta unidad como Delphi banda ancha, y en la industria como planning poker. Resuelve siempre lo mismo: que el primero que habla no arrastre al resto.

24.4 Velocity

La velocity es la cantidad de puntos que el equipo efectivamente completa en un sprint; con el promedio de dos o tres sprints se anticipa cuántos entran en el próximo.

Pero la velocity es una predicción, no una estimación: como contar los autos de los sábados anteriores, usa la historia y no mide atributos del objeto. Los Puntos de Función son estimación; la velocity, predicción. Por eso sirven en momentos distintos:

	Puntos de Función	Velocity
Cuándo	Antes de empezar	Después de 2 o 3 sprints
Necesita	Los requerimientos	La historia del propio equipo
Da	Un tamaño absoluto	Un ritmo del equipo
Sirve para	Cotizar, contratar, comparar proyectos	Planificar el próximo sprint

Sin historia no hay velocity, y el primer día de un proyecto nunca hay historia.

24.5 La trampa de la velocity

Los puntos de un equipo no son comparables con los de otro: cada equipo calibra su propia escala contra su propia historia. Usar la velocity para comparar equipos, o exigir que suba, es exactamente el caso de la sección 9: la métrica se convierte en objetivo y los puntos se inflan.

Las dos técnicas necesitan datos propios: los PF, las tablas de conversión y la productividad de la organización; la velocity, los sprints anteriores del equipo. Y esos datos sólo existen si alguien midió.

25. Síntesis

EL HILO EN UNA FRASE

Se mide para poder estimar, y se estima para poder decidir. Lo que pasó en cada proyecto se registra, porque la próxima estimación se hace con esos datos.

Vuelvan a los números que anotaron al comienzo para su ISFPP. ¿Eran una estimación o una predicción? ¿Un rango o un punto? Con lo visto deberían poder rehacerlos: contar los PF, ajustarlos, convertirlos con supuestos explícitos y entregar un rango.

Preguntas para repasar

- En el caso de los dos equipos, ¿qué entidad, qué atributo y qué normalizador harían falta para decidir cuál trabajó mejor?
- ¿Por qué no tiene sentido promediar severidades «baja, media, alta»? ¿Qué hubo que hacer en la tabla de riesgos de la Unidad 2 para poder multiplicar probabilidad por impacto?
- Construyan una métrica para un atributo de su ISFPP con la receta de cuatro pasos y evalúenla con los cinco criterios. ¿Qué le pasaría si se usara para evaluar a los integrantes del grupo?
- Expliquen la diferencia entre estimar y predecir con la analogía del puente. ¿Cuál de las dos es la velocity de Scrum? ¿Y los Puntos de Función?
- ¿Por qué cinco modelos empíricos dan resultados tan distintos para el mismo sistema? ¿Qué haría falta para que uno de ellos sirva en su organización?
- ¿Por qué conviene separar la estimación del compromiso aunque los decida la misma persona?
- En el sistema de fútbol, ¿cuánto cambiarían los PFSA si se hubieran olvidado las entidades delegado y cancha? ¿Y si el «listado de equipos» se contara como EQ?
- Recorran la cadena PFSA → PFA → LDC → esfuerzo → plazo → costo para su ISFPP e indiquen, en cada paso, qué dato histórico o qué supuesto usaron.

Bibliografía

- Pressman, R. y Maxim, B. *Ingeniería del software: un enfoque práctico*, 9ª ed. McGraw-Hill.
- Sommerville, I. *Ingeniería de software*, 10ª ed. Pearson.
- Fenton, N. y Pfleeger, S. L. *Software Metrics: A Rigorous and Practical Approach*, 2ª ed. PWS, 1997.
- Park, R., Goethert, W. y Florac, W. *Goal-Driven Software Measurement: A Guidebook*. SEI, 1996.
- Gilb, T. *Principles of Software Engineering Management*. Addison-Wesley, 1988.
- Boehm, B. *Software Engineering Economics*. Prentice Hall, 1981.
- Boehm, B. et al. *Software Cost Estimation with COCOMO II*. Prentice Hall, 2000.
- McConnell, S. *Software Estimation: Demystifying the Black Art*. Microsoft Press, 2006.
- Brooks, F. *The Mythical Man-Month*. Addison-Wesley, 1975 (ed. aniversario, 1995).
- Albrecht, A. «Measuring Application Development Productivity». IBM Application Development Symposium, 1979.
- IFPUG. *Function Point Counting Practices Manual*. International Function Point Users Group.
- McCabe, T. «A Complexity Measure». *IEEE TSE*, 1976.
- Chidamber, S. y Kemerer, C. «A Metrics Suite for Object Oriented Design». *IEEE TSE*, 1994.
- Cohn, M. *Agile Estimating and Planning*. Prentice Hall, 2005.
- IEEE Std 610.12-1990. *IEEE Standard Glossary of Software Engineering Terminology*.